

OrientedDiffDet: Complete Technical Survey - Part 1

Based on Source Materials

September 7, 2026

Abstract

This report provides a complete technical survey of OrientedDiffDet, a diffusion-based method for oriented object detection in aerial images. All content, definitions, formulas, derivations, examples, and comparisons are taken directly from the source materials without addition. The survey covers: (1) the problem of oriented object detection in remote-sensing images; (2) the limitations of horizontal bounding boxes; (3) oriented bounding box representation using six parameters $(x, y, w, h, \Delta\alpha, \Delta\beta)$; (4) the four vertices derivation; (5) feature alignment with oriented RoIs; (6) bi-linear interpolation; (7) the diffusion process on bounding boxes; (8) the forward diffusion equation $B_t = \alpha_t B_0 + \sigma_t \epsilon$; (9) the accelerated diffusion ODE solver; (10) the semi-linear ODE formulation; (11) Equation (6) for accelerated sampling; (12) the log-SNR parameterization $\lambda_t = \log(\alpha_t/\sigma_t)$; (13) Backbone + FPN architecture; (14) RoI pooling and feature alignment; (15) detection heads for classification, box refinement, and noise prediction; (16) training vs inference distinction; (17) the complete architecture pipeline; and (18) the core intuition of diffusion-based detection.

Contents

1	Introduction: The Problem with Remote-Sensing Images	7
1.1	Main Idea of the Paper	7
1.2	Problems with Remote-Sensing Images	7
1.3	Oriented Bounding Boxes	7
1.4	Feature Alignment	7
1.5	Diffusion-Based Detection	8
1.6	DPM-Solver / Acceleration	8
1.7	Datasets	8
1.8	Main Contribution in One Sentence	8
1.9	Conceptual Distinction	8
2	The Diffusion Variable: Bounding Boxes	9
2.1	Standard Diffusion Generative Model	9
2.2	Important Distinction for This Paper	9
2.3	Conditional Diffusion	9
2.3.1	Unconditional Diffusion	9
2.3.2	Conditional Diffusion	10

3	Horizontal vs Oriented Bounding Boxes	10
3.1	Horizontal Bounding Box	10
3.2	Oriented Bounding Box	10
3.3	Why This Matters in Remote Sensing	11
4	The Six-Parameter Representation	11
4.1	Why Their Representation is Less Prone to Overfitting	11
4.2	The Feature-Alignment Problem	11
4.3	What Feature Alignment Does	12
4.4	The Simple Picture	12
4.5	Division of Responsibilities	12
5	ROI Pooling and Feature Alignment	13
5.1	What ROI Pooling Actually Does	13
5.2	Does ROI Pooling Remove Background?	13
5.3	Why Feature Alignment is Important	13
5.4	Summary of ROI Pooling and Feature Alignment	14
6	Region Proposal Network (RPN)	14
6.1	What is RPN?	14
6.2	Why the Paper Mentions Rotated RPN	14
7	Anchors vs Diffusion-Based Box Generation	15
7.1	Older Methods Use Prior Anchors	15
7.2	Two-Stage Detectors	15
7.3	Single-Stage Detectors	15
7.4	The Problem with the Angle Parameter	15
7.5	What OrientedDiffDet Changes	16
7.6	Conditional Diffusion Connection	16
8	Anchors vs Features: A Clarification	16
8.1	Anchors Are Not Features	16
8.2	The Number of Anchors is Fixed	17
8.3	Why Diffusion Approach is Attractive	17
8.4	The Real Problem with Anchors	17
9	The Complete Architecture Overview	18
9.1	High-Level View	18
9.2	Internal Process	18
9.3	What Gets Noised	18
9.3.1	During Training	18
9.3.2	During Inference	18
9.4	One-Line Answer	19
10	Backbone + FPN	19
10.1	What is the Backbone?	19
10.2	What is the FPN?	19
10.3	What Backbone + FPN Does	20
10.4	Very Simple Analogy	20

10.5	FPN vs U-Net	20
10.5.1	Basic Distinction	20
10.5.2	U-Net	20
10.5.3	FPN	20
10.5.4	Main Differences	21
10.5.5	Important Conceptual Difference	21
10.5.6	Historical Note	21
11	Horizontal Bounding Boxes Throw Away Orientation Information	22
11.1	The Problem	22
11.2	Why This Happens	22
11.3	Solution: Oriented Bounding Box	22
12	Midpoint-Based Parameterization	23
12.1	What is Midpoint-Based Parameterization?	23
12.2	For an Oriented Bounding Box	23
12.3	The Problem with Angle Regression	23
12.3.1	The Problem	23
12.3.2	Why This is Called a Discontinuity	24
12.3.3	Why Midpoint-Based Parameterization Helps	24
12.4	Representation Ambiguity	24
12.5	Why This Hurts Training	25
12.6	Why This Hurts Training	25
13	The Six-Parameter Oriented Box Representation	26
13.1	The Representation	26
13.2	Parameter Meanings	26
13.3	The Four Vertices	26
13.4	Why This Creates Rotation	26
13.5	Geometrical Interpretation	27
13.6	Geometric Properties	27
14	From Parallelogram to Oriented Rectangle	28
14.1	Why Convert to Oriented Rectangle?	28
14.2	Why Change It?	28
14.3	Extending the Shorter Diagonal	28
14.4	The Reason	29
15	Angle Calculation from Six Parameters	29
15.1	The Relationship	29
15.2	Geometric Derivation	29
15.3	Why θ Depends on $\Delta\alpha, \Delta\beta$	30
15.4	The Complete Chain	30
16	Feature Alignment Mathematics	30
16.1	The Feature Alignment Equation	30
16.2	Breaking Down Each Term	30
16.3	What is $R(x, y, \theta)$?	30
16.4	The Sampling Process	31

16.5 The Rotation Matrix	31
16.6 Why We Need This Transformation	31
17 Bilinear Interpolation	32
17.1 Why Bilinear Interpolation?	32
17.2 Bilinear Interpolation Formula	32
17.3 The Complete Feature Alignment Operation	32
18 The Complete Architecture Pipeline	33
18.1 Complete Architecture Diagram	33
19 The Five Most Important Equations	34
19.1 Equation 1: Oriented Box Representation	34
19.2 Equation 2: Four Vertices	34
19.3 Equation 3: Forward Diffusion	34
19.4 Equation 4: Feature Alignment	35
19.5 Equation 5: Accelerated Diffusion	35
20 Summary of Part 1	35
21 The Diffusion Process on Bounding Boxes	35
21.1 Forward Diffusion Equation	35
21.2 General Forward Diffusion Equation	36
21.3 What α_t Does	36
21.4 The Cosine Schedule	37
21.5 Why the Schedule is Necessary	37
22 Training Process	37
22.1 Training Steps	37
22.2 What the Network Learns	38
22.3 Why the Image Helps	38
23 Inference Process	38
23.1 Starting Point	38
23.2 Reverse Process	38
23.3 Random Boxes Are Not Extra Information	39
24 Reverse Process Mathematics	39
24.1 Forward Process	39
24.2 Reverse Process	39
24.3 Network Noise Prediction	39
24.4 Deriving the Clean Box Estimate	39
24.5 Meaning of the Denoising Network	40
25 Why Many Steps Are Needed	40
26 The Problem: Inference is Slow	40

27 Accelerated Diffusion	40
27.1 The Basic Idea	40
27.2 Diffusion as an ODE	41
27.3 What Semi-Linear Means	41
28 The Paper's Accelerated Diffusion Equation	41
28.1 Equation (6)	41
28.2 What is $\tilde{x}_{t_0}$?	42
28.3 What are t_i ?	42
28.4 What is λ_t ?	42
28.5 Why Use λ Instead of t ?	43
28.6 Understanding $e^{h_i} - 1$	43
28.7 Understanding Equation (6) Intuitively	44
28.7.1 Part 1	44
28.7.2 Part 2	44
28.8 Why the Equation Subtracts the Predicted Noise	44
28.9 Why Can We Skip Many Timesteps?	45
28.10 Why This is Different from Simply Increasing Step Size	45
28.11 What "Exponentially Weighted Integral" Means	45
29 Putting Accelerated Diffusion into Object Detection	45
29.1 Inference with Accelerated Diffusion	45
29.2 The Complete Inference Pipeline	46
30 Training vs Inference Distinction	47
30.1 Training	47
30.2 Training vs Inference Table	47
31 The Four Major Problems and Four Solutions	48
32 The Complete Mathematical Formulation	48
32.1 Step 1: Image Encoding	48
32.2 Step 2: Ground-Truth Oriented Box	48
32.3 Step 3: Generate Vertices	48
32.4 Step 4: Add Diffusion Noise	48
32.5 Step 5: Extract Oriented RoI	49
32.6 Step 6: Feature Alignment	49
32.7 Step 7: Detection Head	49
32.8 Step 8: Noise Prediction	49
32.9 Step 9: Estimate Clean Box	49
32.10 Step 10: Accelerated Update	49
33 The Most Intuitive Interpretation of Equation (6)	50
34 Important Distinction: Feature Alignment vs Diffusion	50

35 Complete Conceptual Picture	50
35.1 Input	50
35.2 Image Encoder	50
35.3 Training	51
35.4 Inference	51
36 Summary of Key Equations	51
37 Conclusion	51

1 Introduction: The Problem with Remote-Sensing Images

This introduction describes a diffusion-based object detection method specifically designed for remote-sensing images.

1.1 Main Idea of the Paper

The paper’s main idea is:

Instead of starting object detection with predefined/candidate bounding boxes and then refining them, it starts with random noisy boxes and uses a diffusion model to gradually transform those random boxes into accurate object boxes.

1.2 Problems with Remote-Sensing Images

There are several important problems with remote-sensing images:

1. Objects can appear at very different scales.
2. Objects are often densely packed.
3. Objects can have arbitrary orientations.
4. Horizontal bounding boxes can therefore contain a lot of unwanted background.

1.3 Oriented Bounding Boxes

Instead of representing an object only as:

$$(x, y, w, h) \tag{1}$$

the method uses six parameters, adding information about the orientation/edges.

This allows objects such as ships, ports, aircraft, etc. to be represented more accurately when rotated.

1.4 Feature Alignment

The paper introduces a feature alignment algorithm to deal with the mismatch between horizontal and oriented regions.

Its purpose is to extract features that are less sensitive to object rotation.

This is particularly important when ROI pooling would otherwise introduce a rounding/alignment error.

1.5 Diffusion-Based Detection

The detection problem is formulated as a generation/denoising problem.

Start with noisy/random bounding boxes:

$$B_T \tag{2}$$

The diffusion model progressively denoises them:

$$B_T \rightarrow B_{T-1} \rightarrow \cdots \rightarrow B_0 \tag{3}$$

The final B_0 represents the predicted object boxes.

1.6 DPM-Solver / Acceleration

A major practical problem with diffusion models is that they can require many denoising steps.

The paper uses DPM-Solver and adjusts the number of sampling/denoising steps to make detection faster while maintaining performance.

1.7 Datasets

The method is evaluated on DOTA and HRSC2016, which are commonly used for oriented object detection in aerial/remote-sensing imagery.

1.8 Main Contribution in One Sentence

The paper essentially combines:

- Oriented object detection
- Feature alignment
- Diffusion-based box generation
- Accelerated diffusion sampling

to detect arbitrarily oriented objects in remote-sensing images.

1.9 Conceptual Distinction

The most important conceptual distinction to remember is:

Traditional detector: candidate/proposal boxes $\rightarrow$ box regression $\rightarrow$ final boxes

This paper: random/noisy boxes $\rightarrow$ diffusion denoising $\rightarrow$ final object boxes

2 The Diffusion Variable: Bounding Boxes

2.1 Standard Diffusion Generative Model

For a standard diffusion generative model, during inference you typically start from pure random noise and progressively denoise it to generate the output.

For an image diffusion model:

$$x_T \sim \mathcal{N}(0, I) \quad (4)$$

Then:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_1 \rightarrow x_0 \quad (5)$$

where x_0 is the generated image.

2.2 Important Distinction for This Paper

They are using diffusion for object detection, not directly generating an image.

So the diffusion variable is the bounding box, not the image.

Conceptually:

$$\boxed{\text{random/noisy boxes} \rightarrow \text{denoising steps} \rightarrow \text{object bounding boxes}} \quad (6)$$

For example, suppose an image contains a ship. At inference, the model might start with random box coordinates such as:

$$(x, y, w, h, \dots)_{\text{noisy}} \quad (7)$$

and progressively refine them until they become something like:

$$(x, y, w, h, \theta)_{\text{object}} \quad (8)$$

So yes, the basic idea is: start from noise and generate/refine the desired output. The key is that in this paper, the noise is applied to the box representation rather than generating the entire remote-sensing image.

2.3 Conditional Diffusion

Conditional diffusion is a very good way to describe what is happening here.

2.3.1 Unconditional Diffusion

You start with noise and generate an output without any external information:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0 \quad (9)$$

For image generation:

$$\text{noise} \rightarrow \text{generated image} \quad (10)$$

2.3.2 Conditional Diffusion

You still start from noisy data, but the denoising process is guided by some condition c :

$$x_T \xrightarrow[\text{condition } c]{} x_{T-1} \rightarrow \cdots \rightarrow x_0 \quad (11)$$

In this paper, the condition is essentially the remote-sensing image/features. So conceptually:

$$\boxed{\text{random/noisy boxes} + \text{image features} \rightarrow \text{denoised object boxes}} \quad (12)$$

The model asks:

Given this remote-sensing image, how should I progressively modify these noisy/random boxes so that they become the boxes of the objects present in the image?

So you can think of it as:

$$\text{Image} \rightarrow \text{condition/features} \rightarrow \text{diffusion process} \rightarrow \text{object boxes} \quad (13)$$

rather than:

$$\text{Noise} \rightarrow \text{image} \quad (14)$$

3 Horizontal vs Oriented Bounding Boxes

3.1 Horizontal Bounding Box

A horizontal box is constrained to remain parallel to the image axes. The ship is rotated, but the box cannot rotate with it. Therefore, a lot of the box contains background.

3.2 Oriented Bounding Box

An oriented box can rotate. Now the box follows the ship's actual orientation, so it is much tighter.

Mathematically, instead of only:

$$(x_c, y_c, w, h) \quad (15)$$

you can represent it as:

$$(x_c, y_c, w, h, \theta) \quad (16)$$

where θ is the rotation angle.

3.3 Why This Matters in Remote Sensing

Imagine a satellite image containing many ships. If a ship is at 45° , a horizontal box may cover a large amount of sea around it. An oriented box can rotate 45° and tightly surround the ship.

So the paper's argument is:

Remote-sensing objects can have arbitrary orientations $\rightarrow$ horizontal boxes may contain unnecessary background $\rightarrow$ use oriented boxes $\rightarrow$ better localization and potentially better classification.

And this is separate from the diffusion idea. Diffusion determines how the boxes are generated/refined; oriented boxes determine what kind of box representation is being generated.

4 The Six-Parameter Representation

4.1 Why Their Representation is Less Prone to Overfitting

Instead of using a rotated RPN, they modify their regression output from 4 parameters to 6 parameters.

A normal horizontal box might be:

$$(x_c, y_c, w, h) \tag{17}$$

Their oriented representation adds two parameters describing the oriented edges, so it becomes 6 parameters.

The advantage is that this is implemented using a lightweight fully convolutional network, rather than introducing a more complicated rotated RPN with many additional parameters.

So:

$$\text{fewer parameters} \rightarrow \text{less model complexity} \rightarrow \text{lower risk of overfitting} \tag{18}$$

4.2 The Feature-Alignment Problem

Suppose the actual object is rotated, but the feature extractor initially has a horizontal ROI. The horizontal ROI contains the object, but also unnecessary background.

When we perform ROI pooling, the continuous coordinates of the ROI have to be mapped onto the discrete feature-map grid. That mapping involves rounding/alignment, which can cause the pooled features to be slightly shifted or distorted.

This is what they mean by the:

"rounding effect on the horizontal RoI caused by pooling."

4.3 What Feature Alignment Does

They essentially learn a transformation that says:

"This horizontal region corresponds to a rotated object. Let me transform/align the features so that they correspond better to the object's actual orientation."

Conceptually:

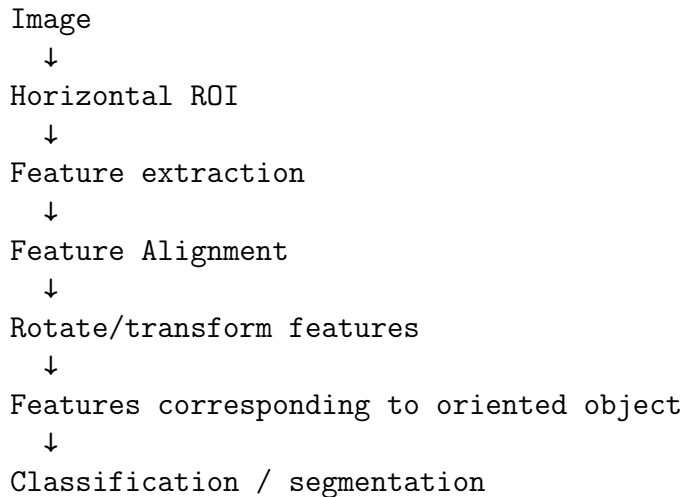
$$\boxed{\text{Horizontal ROI} \xrightarrow{\text{learned transformation}} \text{Oriented ROI}} \quad (19)$$

Then the network can extract features from the object in a way that is less sensitive to its rotation.

That's why they say:

"extract rotation-invariant features from oriented boxes."

4.4 The Simple Picture



So feature alignment is not changing the physical object or simply rotating the image. It is learning how to transform the extracted features so that the network can properly represent the rotated object.

4.5 Division of Responsibilities

The feature alignment module is not completely removing rotation from the problem so that the diffusion model can ignore it. Rather, it helps compensate for the rotation and the spatial misalignment caused by ROI pooling.

Think of the responsibilities like this:

- **Oriented box representation:** explicitly represents the object's rotation.
- **Feature alignment:** aligns/transforms the features so that the object's rotated structure is represented properly and background/misalignment is reduced.
- **Diffusion model:** learns to progressively transform noisy boxes into the correct object boxes.

So the division is roughly:

$$\boxed{\text{Feature Alignment} \rightarrow \text{better rotation-aligned features}} \quad (20)$$

$$\boxed{\text{Diffusion} \rightarrow \text{generate/refine the correct oriented box}} \quad (21)$$

5 ROI Pooling and Feature Alignment

5.1 What ROI Pooling Actually Does

Suppose your CNN produces a large feature map. The detector has a Region of Interest (ROI) corresponding to the suspected object.

ROI pooling takes that region from the feature map and converts it into a fixed-size feature representation, regardless of the ROI's original size.

For example:

$$\text{ROI of size } 20 \times 40 \rightarrow 7 \times 7 \text{ feature} \quad (22)$$

This is useful because different objects have different sizes, but the subsequent network expects a fixed-size input.

5.2 Does ROI Pooling Remove Background?

It reduces the amount of irrelevant image information being processed, because instead of using the entire image for that object, we focus on a selected region.

But the ROI can still contain background. For example, with a horizontal box around a rotated ship, ROI pooling pools everything inside that rectangle, including the sea.

So it doesn't say "remove all the background." It says "extract a fixed-size feature representation from this selected region."

5.3 Why Feature Alignment is Important

For a rotated object, the horizontal ROI can contain substantial background:

$$\text{Horizontal ROI} = \text{Object} + \text{Background} \quad (23)$$

After pooling, the features can also suffer from spatial quantization/rounding.

The feature-alignment module tries to transform the features so that they correspond better to the oriented object, rather than being poorly aligned because of the horizontal ROI.

So the chain is:

$$\boxed{\text{Image} \rightarrow \text{ROI} \rightarrow \text{ROI pooling} \rightarrow \text{features} \rightarrow \text{feature alignment}} \quad (24)$$

ROI pooling = fixed-size feature extraction.

Feature alignment = better spatial/orientation alignment of those features.

5.4 Summary of ROI Pooling and Feature Alignment

- ROI pooling: takes the features inside a selected ROI and converts them to a fixed-size representation. Because it focuses on the ROI rather than the whole image, it can reduce irrelevant background, but it does not explicitly remove background noise.
- Feature alignment: transforms/aligns those ROI features so that they better correspond to the actual oriented object. This helps with object rotation, spatial misalignment, quantization/rounding effects, and the unwanted influence of background in a horizontal ROI.

So a very good way to remember it is:

$$\boxed{\text{ROI Pooling} \rightarrow \text{fixed-size features}} \quad (25)$$

$$\boxed{\text{Feature Alignment} \rightarrow \text{better aligned, rotation-robust object features}} \quad (26)$$

And together:

$$\text{Image} \rightarrow \text{ROI} \rightarrow \text{fixed-size features} \rightarrow \text{feature alignment} \rightarrow \text{better object representation} \quad (27)$$

6 Region Proposal Network (RPN)

6.1 What is RPN?

A Region Proposal Network (RPN) proposes candidate bounding boxes that may contain objects.

In a two-stage detector such as Faster R-CNN, the basic flow is:

$$\boxed{\text{Image} \rightarrow \text{CNN feature map} \rightarrow \text{RPN} \rightarrow \text{candidate bounding boxes} \rightarrow \text{ROI pooling} \rightarrow \text{classification} + \text{bbox regression}} \quad (28)$$

The RPN doesn't necessarily give the final accurate boxes. It gives candidate/proposal boxes.

6.2 Why the Paper Mentions Rotated RPN

A conventional RPN generally proposes horizontal boxes:

$$(x, y, w, h) \quad (29)$$

For remote-sensing objects, a rotated RPN can instead propose oriented boxes, which additionally represent the object's orientation.

The paper is saying that rather than adding a relatively parameter-heavy rotated RPN, they use a lightweight convolutional approach to produce the six box parameters.

So:

RPN $\rightarrow$ proposes bounding boxes (30)

ROI pooling $\rightarrow$ extracts fixed-size features from those proposed regions (31)

Feature alignment $\rightarrow$ makes those features better aligned with the actual oriented object (32)

7 Anchors vs Diffusion-Based Box Generation

7.1 Older Methods Use Prior Anchors

Traditional object detectors often start with predefined anchor boxes. For oriented detection, you need anchors that account for different positions, sizes, aspect ratios, and orientations.

So you might have many predefined rotated anchors:

$$\text{anchors} = (x, y, w, h, \theta) \quad (33)$$

The detector then learns to adjust these anchors to match the actual objects.

7.2 Two-Stage Detectors

Methods such as ICN, Gliding Vertex, and SCRDet are two-stage approaches:

Image $\rightarrow$ candidate regions/boxes $\rightarrow$ refinement $\rightarrow$ final boxes

 (34)

They can achieve good accuracy, but the multiple stages and complex structures make them computationally expensive.

7.3 Single-Stage Detectors

Methods such as R3Det and RSDet try to make detection faster:

Image $\rightarrow$ prediction $\rightarrow$ boxes

 (35)

They are more efficient and therefore attractive for things like edge devices, but there can be a trade-off in accuracy.

7.4 The Problem with the Angle Parameter

The angle θ is a periodic variable. For example, $\theta = 179^\circ$ and $\theta = -179^\circ$ represent almost the same orientation, but numerically they are very far apart. That can make angle regression sensitive and difficult.

7.5 What OrientedDiffDet Changes

This is the key sentence:

"We directly generate rotated detection boxes from a set of random rotated boxes using the diffusion model, eliminating the need for obtaining candidate anchors from prior knowledge."

Instead of:

$$\boxed{\text{predefined rotated anchors} \rightarrow \text{regression} \rightarrow \text{final boxes}} \quad (36)$$

they propose:

$$\boxed{\text{random rotated boxes} \rightarrow \text{diffusion denoising} \rightarrow \text{object boxes}} \quad (37)$$

So the model doesn't need to decide beforehand: "I need anchors at these particular sizes, aspect ratios, positions, and angles."

Instead, it starts with random boxes and learns to transform them toward the actual objects.

7.6 Conditional Diffusion Connection

Here, you can think of it as:

$$\boxed{\text{random boxes} + \text{image features} \rightarrow \text{correct oriented boxes}} \quad (38)$$

The image provides the condition, while the diffusion process generates/refines the boxes.

So the paper's central motivation is essentially:

Why rely on hand-designed prior anchors for rotated objects when we can learn to generate the required oriented boxes directly through a diffusion process?

8 Anchors vs Features: A Clarification

8.1 Anchors Are Not Features

Anchors are not features. They are predefined candidate bounding boxes used as starting points for detecting objects.

Think of the distinction:

- **Features** = information extracted from the image by the CNN, such as edges, textures, shapes, etc.
- **Anchors** = predefined boxes placed at different locations, sizes, aspect ratios, and sometimes angles.

So:

$$\boxed{\text{Image} \rightarrow \text{features}} \quad (39)$$

and separately:

$$\boxed{\text{anchors} \rightarrow \text{candidate boxes}} \quad (40)$$

Then the features are used to determine which anchors correspond to objects and how they should be adjusted.

8.2 The Number of Anchors is Fixed

The number of anchor boxes is fixed once the detector architecture and anchor configuration are defined.

For example, suppose a detector has 100×100 spatial locations and 3 anchor sizes/aspect ratios per location. Then it has:

$$100 \times 100 \times 3 = 30,000 \quad (41)$$

predefined anchor boxes at that feature level.

For an oriented detector, you may additionally have several predefined angles, making the number even larger.

8.3 Why Diffusion Approach is Attractive

The important point is that anchors are fixed in number and predefined in their initial geometry; the network learns how to adjust them to match actual objects.

This is one motivation for the diffusion approach:

$$\text{many predefined anchors} \rightarrow \text{random boxes} + \text{diffusion generation} \quad (42)$$

So the authors avoid depending on a manually designed set of anchor boxes.

8.4 The Real Problem with Anchors

The bigger problem is: how do you choose a good set of anchors beforehand?

Remote-sensing images can contain very small objects, very large objects, long/narrow objects, different aspect ratios, and arbitrary orientations.

So you would need many anchors to cover these possibilities. That can lead to:

$$\boxed{\text{more anchors} \rightarrow \text{more computation} + \text{complexity}} \quad (43)$$

and potentially poor coverage if the predefined anchors don't match the objects well. That's why OrientedDiffDet's idea is attractive:

$$\boxed{\text{predefined anchors} \longrightarrow \text{random boxes}} \quad (44)$$

and then:

$$\boxed{\text{random boxes} \xrightarrow{\text{conditional diffusion}} \text{appropriate object boxes}} \quad (45)$$

9 The Complete Architecture Overview

9.1 High-Level View

The basic idea is:

$$\boxed{\text{Input: Remote-sensing image} \rightarrow \text{Image feature extraction} + \text{conditional diffusion} \rightarrow \text{Output: Oriented boxes}} \quad (46)$$

For example, if the image contains three ships:

$$\text{Image} \rightarrow \{B_1, B_2, B_3\} \quad (47)$$

where each oriented box contains information about the object's location, size, and orientation.

9.2 Internal Process

The diffusion model itself doesn't simply take the raw image and directly output a box in one step. Internally, it starts with random rotated/noisy boxes, and the image features condition the diffusion process:

$$\boxed{\text{Image} \rightarrow \text{image features}} \quad (48)$$

and:

$$\boxed{\text{Random rotated boxes} \xrightarrow[\text{image features}]{\text{diffusion}} \text{correct oriented boxes}} \quad (49)$$

9.3 What Gets Noised

For this object-detection paper, you noise the bounding boxes.

9.3.1 During Training

You have:

$$\boxed{\text{Image} + \text{Ground-truth oriented bounding box}} \quad (50)$$

You take the ground-truth box and add diffusion noise to it:

$$\boxed{B_0 \rightarrow B_t} \quad (51)$$

The image is not noised. The image is used to provide the conditioning/features.

9.3.2 During Inference

You don't have the ground-truth box, so you start with a random/noisy box:

$$\boxed{B_T} \quad (52)$$

Then condition the denoising on the image:

$$\boxed{\text{Image features} + B_T \rightarrow B_{T-1} \rightarrow \dots \rightarrow B_0} \quad (53)$$

where B_0 is the detected oriented bounding box.

9.4 One-Line Answer

In this object-detection diffusion model, we noise the bounding boxes, not the image; the image acts as the condition that guides the noisy boxes toward the correct object boxes.

10 Backbone + FPN

10.1 What is the Backbone?

The backbone is the main feature-extraction network. It takes the raw image:

$$\boxed{\text{Image} \rightarrow \text{Backbone} \rightarrow \text{Feature maps}} \quad (54)$$

For example, a backbone could be ResNet-50, ResNet-101, Swin Transformer, etc. Its job is to gradually understand what is present in the image:

- Early layers $\rightarrow$ edges, colors, simple shapes
- Middle layers $\rightarrow$ textures, parts, patterns
- Deep layers $\rightarrow$ high-level semantic information, such as "this region probably contains an airplane"

So you can think:

Backbone = the eyes of the model that extract visual information.

10.2 What is the FPN?

FPN stands for Feature Pyramid Network.

The problem is that objects in an image can have different sizes. The backbone produces feature maps at different resolutions, but deep feature maps are very good at understanding what an object is, while shallow feature maps preserve more fine spatial details.

The FPN combines information from different levels.

Conceptually:

$$\text{Backbone} \rightarrow \begin{cases} \text{high resolution features} \\ \text{medium resolution features} \\ \text{low resolution features} \end{cases} \quad (55)$$

and FPN combines them into:

$$\boxed{\text{Multi-scale feature maps}} \quad (56)$$

10.3 What Backbone + FPN Does

The complete process is roughly:

$$\boxed{\text{Input Image} \rightarrow \text{Backbone} \rightarrow \text{FPN} \rightarrow \text{Feature Maps}} \quad (57)$$

- Backbone extracts visual features at different depths.
- FPN combines these features so the model has useful information at multiple scales.
- These feature maps are then given to the detection decoder.

10.4 Very Simple Analogy

Imagine looking at a satellite image.

Backbone: "I can see roads, buildings, vehicles, fields, etc."

FPN: "I will organize information at different scales so that both a huge building and a tiny vehicle can be detected."

Detection decoder: "Now I'll use these features to determine exactly where the objects are."

So in this paper:

$$\boxed{\text{Image} \rightarrow \text{Backbone} + \text{FPN} \rightarrow \text{Feature Maps} \rightarrow \text{RoI features} \rightarrow \text{Detection Head} \rightarrow \text{Refined Boxes}} \quad (58)$$

10.5 FPN vs U-Net

10.5.1 Basic Distinction

The core idea is similar, but their architecture and purpose are different.

10.5.2 U-Net

U-Net has a complete encoder-decoder structure. The decoder progressively upsamples and uses skip connections from the encoder. The goal is to recover a detailed, high-resolution representation for segmentation.

10.5.3 FPN

FPN usually sits on top of a backbone such as ResNet. The backbone gives C_3, C_4, C_5 . Then FPN creates P_3, P_4, P_5 using a top-down pathway.

FPN does not necessarily reconstruct the original image resolution. Instead, it creates useful feature maps at multiple scales.

10.5.4 Main Differences

Table 1: U-Net vs FPN Comparison

U-Net	FPN
Encoder + decoder Segmentation	Backbone + pyramid Multi-scale feature representation, especially detection
Yes skip/lateral connections	Yes skip/lateral connections
Yes upsampling	Yes upsampling
Usually concatenation	Usually addition after lateral projection
Usually one high-resolution segmentation map	Multiple feature maps $P_3, P_4, P_5, \dots$
Usually yes reconstructs image resolution	Not necessarily
Yes can use pretrained backbone	Very commonly

10.5.5 Important Conceptual Difference

$$\boxed{\text{U-Net: Compress information} \rightarrow \text{recover detailed high-resolution information}} \quad (59)$$

$$\boxed{\text{FPN: Create useful representations at multiple scales}} \quad (60)$$

10.5.6 Historical Note

U-Net (2015) predates the Feature Pyramid Network paper (2016). So U-Net could not have been designed based on the later FPN architecture.

The relationship is better stated as:

U-Net and FPN use a similar multi-scale feature-fusion idea.

Both use:

- deep features $\rightarrow$ high semantic/context information
- shallow features $\rightarrow$ high spatial resolution
- upsampling
- lateral/skip connections
- combining the two

But their goals differ:

$$\boxed{\text{U-Net} \rightarrow \text{high-resolution segmentation}} \quad (61)$$

$$\boxed{\text{FPN} \rightarrow \text{multi-scale feature maps, especially for detection}} \quad (62)$$

11 Horizontal Bounding Boxes Throw Away Orientation Information

11.1 The Problem

A normal horizontal bounding box does not tell you the object's orientation.

For example, imagine a long ship. Two objects could have nearly the same horizontal bounding box:

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max}) \tag{63}$$

even though one ship is oriented like " / " and the other like " \ ":

11.2 Why This Happens

A horizontal bounding box (HBB) is constrained to be horizontal:

$$\boxed{(x_{\min}, y_{\min}, x_{\max}, y_{\max})} \tag{64}$$

It only describes:

- where the object is
- its width
- its height

It doesn't contain an angle.

So:

$$\boxed{\text{HBB} \not\Rightarrow \text{orientation}} \tag{65}$$

11.3 Solution: Oriented Bounding Box

Instead of a horizontal rectangle, use a rotated/oriented bounding box (OBB):

$$\boxed{(x, y, w, h, \theta)} \tag{66}$$

where θ represents the object's orientation angle.

So the key idea is:

Horizontal bounding boxes throw away the object's rotation information. Two objects with very different orientations can therefore have nearly identical HBBs.

12 Midpoint-Based Parameterization

12.1 What is Midpoint-Based Parameterization?

Midpoint-based parameterization means representing a bounding box using its center (midpoint) rather than only its corner coordinates.

For a normal horizontal bounding box:

$$(x_{\min}, y_{\min}, x_{\max}, y_{\max}) \quad (67)$$

the midpoint is:

$$x_c = \frac{x_{\min} + x_{\max}}{2}, \quad y_c = \frac{y_{\min} + y_{\max}}{2} \quad (68)$$

Then instead of describing the box using four corner coordinates, we can describe it using:

$$\boxed{(x_c, y_c, w, h)} \quad (69)$$

where:

- x_c, y_c = center/midpoint
- w = width
- h = height

12.2 For an Oriented Bounding Box

If the box can rotate, we add an angle:

$$\boxed{(x_c, y_c, w, h, \theta)} \quad (70)$$

So midpoint-based parameterization essentially says:

Represent the object using the center point of its bounding box, together with its size (and, for an oriented box, its angle).

12.3 The Problem with Angle Regression

The problem is not that (x, y, w, h, θ) cannot represent an oriented box. It can. The problem is that θ is a bad quantity to regress directly because angles are periodic.

12.3.1 The Problem

Suppose the angle is represented in the range $-90^\circ \leq \theta < 90^\circ$.

Now imagine two predictions:

$$\theta_1 = 89^\circ \quad (71)$$

and:

$$\theta_2 = -89^\circ \quad (72)$$

Numerically, the difference is:

$$|89 - (-89)| = 178^\circ \quad (73)$$

A neural network using ordinary angle regression would think these two predictions are extremely far apart. But physically, the two rectangles can be almost identically oriented.

12.3.2 Why This is Called a Discontinuity

Imagine increasing the angle continuously:

$$85^\circ, 87^\circ, 89^\circ, 90^\circ, -89^\circ, -87^\circ, \dots \quad (74)$$

Physically, the orientation changes smoothly. But numerically, $89^\circ \rightarrow -89^\circ$ looks like a huge jump of 178° .

So the representation has an artificial discontinuity at the boundary.

12.3.3 Why Midpoint-Based Parameterization Helps

Instead of directly predicting an angle, some oriented detection methods represent the box using points/midpoints or other geometric quantities.

For example, rather than saying:

$$(x, y, w, h, \theta) \quad (75)$$

you can describe the geometry through points such as:

$$(x_1, y_1), \quad (x_2, y_2) \quad (76)$$

from which the orientation can be obtained geometrically.

Then the network is regressing coordinates, rather than having to learn the strange wrap-around behavior of an angle.

So the fundamental issue is:

$$\boxed{\text{Direct angle regression} \rightarrow \text{periodic discontinuity}} \quad (77)$$

whereas a suitable geometric parameterization can make the regression more continuous and easier to learn.

12.4 Representation Ambiguity

The issue is that an oriented bounding box has multiple parameter values that can describe the same physical box.

For example, for a rectangular object:

$$(x, y, w, h, 90^\circ) \quad (78)$$

and:

$$(x, y, w, h, -90^\circ) \quad (79)$$

can correspond to the same physical rectangle, because rotating a rectangle by 180° brings it back to the same geometry.

There is another common ambiguity too:

$$(w, h, \theta) \tag{80}$$

can sometimes be represented equivalently by:

$$(h, w, \theta + 90^\circ) \tag{81}$$

So the problem is essentially:

Different parameter values $\longrightarrow$ same physical bounding box

(82)

12.5 Why This Hurts Training

The fact that two parameterizations can describe the same box is not itself the problem. The problem is what happens when the regression loss compares the predicted parameters to the ground truth.

Suppose the ground truth box is represented as:

$$\theta_{\text{GT}} = 89^\circ \tag{83}$$

but the network predicts:

$$\theta_{\text{pred}} = -89^\circ \tag{84}$$

Geometrically, these boxes are almost the same orientation. But a normal regression loss sees:

$$|\theta_{\text{pred}} - \theta_{\text{GT}}| = |-89 - 89| = 178^\circ \tag{85}$$

So the model receives a huge angle error, even though the predicted box is actually very close to the correct box.

That's the real problem:

Small geometric error $\neq$ small parameter error

(86)

near the angle boundary.

12.6 Why This Hurts Training

The model is told: "Your prediction is 178° wrong!" when visually/geometrically it is almost correct. Therefore, the gradient from the angle loss can push the network in the wrong direction or with unnecessarily large magnitude.

So the issue isn't simply "two bounding boxes become the same." It's:

The same or nearly the same physical bounding box can have very different numerical angle values, causing an ordinary regression loss to report a large error.

That's why angle-based parameterization can be problematic.

13 The Six-Parameter Oriented Box Representation

13.1 The Representation

The paper defines:

$$\boxed{(x, y, w, h, \Delta\alpha, \Delta\beta)} \quad (87)$$

instead of only (x, y, w, h) .

13.2 Parameter Meanings

- x, y : center of the outer horizontal rectangle
- w : horizontal extent
- h : vertical extent
- $\Delta\alpha$: horizontal offset controlling tilt
- $\Delta\beta$: vertical offset controlling tilt

13.3 The Four Vertices

The four vertices of the oriented quadrilateral are:

$$\boxed{P_1 = \left(x + \Delta\alpha, y - \frac{h}{2}\right)} \quad (88)$$

$$\boxed{P_2 = \left(x + \frac{w}{2}, y + \Delta\beta\right)} \quad (89)$$

$$\boxed{P_3 = \left(x - \Delta\alpha, y + \frac{h}{2}\right)} \quad (90)$$

$$\boxed{P_4 = \left(x - \frac{w}{2}, y - \Delta\beta\right)} \quad (91)$$

13.4 Why This Creates Rotation

Start with $\Delta\alpha = 0, \Delta\beta = 0$.

Then:

$$P_1 = \left(x, y - \frac{h}{2}\right) \quad (92)$$

$$P_2 = \left(x + \frac{w}{2}, y\right) \quad (93)$$

$$P_3 = \left(x, y + \frac{h}{2}\right) \quad (94)$$

$$P_4 = \left(x - \frac{w}{2}, y\right) \quad (95)$$

This gives the normal axis-aligned representation according to the paper’s midpoint-based parameterization.

Now suppose $\Delta\alpha > 0$. Then the top point moves right: $x + \Delta\alpha$. At the same time, the opposite bottom point moves left: $x - \Delta\alpha$.

Similarly, $\Delta\beta$ shifts the right and left points vertically.

Thus the rectangle becomes tilted.

13.5 Geometrical Interpretation

You can think of the representation as:

$$\boxed{\underbrace{x, y}_{\text{where}} + \underbrace{w, h}_{\text{size}} + \underbrace{\Delta\alpha, \Delta\beta}_{\text{orientation/deformation}}} \quad (96)$$

Table 2: Six-Parameter Meanings

Parameter	Meaning
x	center x-coordinate
y	center y-coordinate
w	horizontal size
h	vertical size
$\Delta\alpha$	horizontal offset controlling tilt
$\Delta\beta$	vertical offset controlling tilt

13.6 Geometric Properties

The four points form a parallelogram-like oriented proposal.

Notice the symmetry:

$$P_1 + P_3 = (2x, 2y) \quad (97)$$

because:

$$(x + \Delta\alpha) + (x - \Delta\alpha) = 2x \quad (98)$$

and:

$$\left(y - \frac{h}{2}\right) + \left(y + \frac{h}{2}\right) = 2y \quad (99)$$

Therefore the midpoint of P_1P_3 is:

$$\frac{P_1 + P_3}{2} = (x, y) \quad (100)$$

Similarly:

$$P_2 + P_4 = (2x, 2y) \quad (101)$$

so the midpoint of P_2P_4 is also (x, y) .

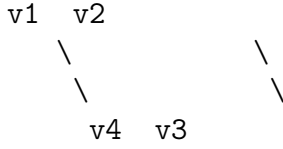
Thus the box remains centered at (x, y) .

14 From Parallelogram to Oriented Rectangle

14.1 Why Convert to Oriented Rectangle?

The parallelogram obtained from the proposal points is not necessarily a proper rectangle, and the next feature-extraction step is easier to define using a regular oriented rectangle.

The four vertices give a parallelogram:



The opposite sides are parallel, but the angles don't necessarily have to be 90° .

14.2 Why Change It?

For RoI feature extraction, we want a clean parameterization:

$$\boxed{(x, y, w, h, \theta)} \quad (102)$$

where:

- x, y = center
- w = width
- h = height
- θ = orientation

A rectangle gives us a clear width, height, center, and orientation.

14.3 Extending the Shorter Diagonal

Suppose the parallelogram has two diagonals:

$$d_{\text{short}} < d_{\text{long}} \quad (103)$$

The paper extends the shorter one so that:

$$d_{\text{short}} = d_{\text{long}} \quad (104)$$

This makes the geometry correspond to a proper oriented rectangle.

14.4 The Reason

$$\boxed{\text{Parallelogram} \rightarrow \text{regular oriented rectangle} \rightarrow \text{consistent RoI sampling}} \quad (105)$$

The resulting rectangle still preserves the important information:

- center/location (x, y)
- overall size (w, h)
- orientation θ
- approximate spatial extent of the object

The exact positions of the four original vertices can change because the shorter diagonal is extended.

But the purpose is not to perfectly preserve every vertex. It is to obtain a stable, well-defined region for feature extraction.

Think of it as:

$$\boxed{\text{preserve important spatial information} + \text{regularize the proposal}} \quad (106)$$

rather than:

$$\boxed{\text{preserve every exact spatial detail}} \quad (107)$$

15 Angle Calculation from Six Parameters

15.1 The Relationship

The paper uses:

$$\boxed{\theta = \tan^{-1} \left(\frac{\frac{h}{2} - \Delta\beta}{\frac{w}{2} + \Delta\alpha} \right)} \quad (108)$$

or a similar ratio depending on the sign convention.

15.2 Geometric Derivation

For any line, if its horizontal displacement is Δx and vertical displacement is Δy , then:

$$\tan \theta = \frac{\Delta y}{\Delta x} \quad (109)$$

Therefore:

$$\boxed{\theta = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right)} \quad (110)$$

15.3 Why θ Depends on $\Delta\alpha, \Delta\beta$

The original horizontal rectangle has no rotation: $\theta = 0$.

When the offsets $\Delta\alpha, \Delta\beta$ are introduced, they move the corresponding points. Therefore the sides/diagonals are tilted.

The amount of tilt depends on the ratio:

$$\frac{\text{vertical displacement}}{\text{horizontal displacement}} \quad (111)$$

So the six-parameter representation implicitly contains the orientation angle.

15.4 The Complete Chain

The conceptual chain is:

$$(w, h, \Delta\alpha, \Delta\beta) \rightarrow \theta \rightarrow R(\theta) \rightarrow \text{rotated sampling points} \quad (112)$$

16 Feature Alignment Mathematics

16.1 The Feature Alignment Equation

The paper gives:

$$F'_c(i, j) = \frac{1}{n} \sum_{(x, y) \in \text{area}(i, j)} F_c(R(x, y, \theta)) \quad (113)$$

16.2 Breaking Down Each Term

- F_c : the c -th channel of the feature map
- $F'_c(i, j)$: aligned feature value at output grid location (i, j)
- $\text{area}(i, j)$: all sampling locations associated with output cell (i, j)
- $R(x, y, \theta)$: rotation/transformation of sampling location according to θ
- n : number of sampled points in that cell
- $\frac{1}{n} \sum$: average pooling over the sampled points

16.3 What is $R(x, y, \theta)$?

$R(x, y, \theta)$ means that the sampling location (x, y) is transformed according to the orientation θ .

In other words, we don't simply take the feature at horizontal coordinates (x, y) . We account for the fact that the RoI is rotated by θ .

Conceptually:

$$\text{point in aligned RoI} \xrightarrow{\text{rotate by } \theta} \text{location in original feature map} \quad (114)$$

16.4 The Sampling Process

The whole process can be thought of as:

$$\boxed{\text{Oriented proposal} \rightarrow \text{divide it into fixed grid} \rightarrow \text{sample points in each grid cell} \rightarrow \text{rotate/transform the points}} \quad (115)$$

The result is:

$$F' \in \mathbb{R}^{m \times m \times C} \quad (116)$$

For example, with $m = 7$:

$$F' \in \mathbb{R}^{7 \times 7 \times C} \quad (117)$$

16.5 The Rotation Matrix

The standard 2-D rotation matrix is:

$$\boxed{R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}} \quad (118)$$

Therefore:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\theta) \begin{bmatrix} u \\ v \end{bmatrix} \quad (119)$$

Expanding:

$$\boxed{x' = u \cos \theta - v \sin \theta} \quad (120)$$

$$\boxed{y' = u \sin \theta + v \cos \theta} \quad (121)$$

Then we translate this to the proposal center:

$$\boxed{x_{\text{sample}} = x_c + u \cos \theta - v \sin \theta} \quad (122)$$

$$\boxed{y_{\text{sample}} = y_c + u \sin \theta + v \cos \theta} \quad (123)$$

16.6 Why We Need This Transformation

Imagine the object is horizontal: $\theta = 0$. Then $\cos 0 = 1, \sin 0 = 0$. Therefore:

$$x_{\text{sample}} = x_c + u \quad (124)$$

$$y_{\text{sample}} = y_c + v \quad (125)$$

So we're just sampling normally.

But suppose $\theta = 45^\circ$. Then $\cos 45^\circ = \sin 45^\circ \approx 0.707$.

Now:

$$x_{\text{sample}} = x_c + 0.707u - 0.707v \quad (126)$$

$$y_{\text{sample}} = y_c + 0.707u + 0.707v \quad (127)$$

Thus the sampling grid itself rotates. That's the essence of feature alignment.

17 Bilinear Interpolation

17.1 Why Bilinear Interpolation?

The rotated sampling point (x_s, y_s) will generally be fractional.

For example:

$$(x_s, y_s) = (32.4, 17.7) \quad (128)$$

The feature map only has discrete pixels. So the model uses the four neighboring points:

$$(32, 17), (33, 17), (32, 18), (33, 18) \quad (129)$$

17.2 Bilinear Interpolation Formula

Suppose the four neighboring feature values are $F_{00}, F_{10}, F_{01}, F_{11}$. Let the fractional coordinates be $a = x - x_0, b = y - y_0$.

Then bilinear interpolation is:

$$\boxed{F(x, y) = (1 - a)(1 - b)F_{00} + a(1 - b)F_{10} + (1 - a)bF_{01} + abF_{11}} \quad (130)$$

17.3 The Complete Feature Alignment Operation

Starting from:

$$(x, y, w, h, \Delta\alpha, \Delta\beta) \quad (131)$$

we obtain the oriented geometry.

Then:

$$\boxed{\theta = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right)} \quad (132)$$

where $\Delta x, \Delta y$ are determined from the oriented proposal geometry.

Then for every local RoI coordinate (u, v) :

$$\boxed{x_s = x_c + u \cos \theta - v \sin \theta} \quad (133)$$

$$\boxed{y_s = y_c + u \sin \theta + v \cos \theta} \quad (134)$$

Then:

$$F_c(x_s, y_s) \quad \text{via bilinear interpolation} \quad (135)$$

and finally average the samples:

$$F'_c(i, j) = \frac{1}{n} \sum_{(x, y) \in \text{area}(i, j)} F_c(R(x, y, \theta)) \quad (136)$$

So the entire operation is:

$$\boxed{\text{oriented box} \rightarrow \theta \rightarrow \text{rotate sampling grid} \rightarrow \text{bilinear interpolation} \rightarrow \text{average} \rightarrow 7 \times 7 \times C \text{ aligned features}} \quad (137)$$

18 The Complete Architecture Pipeline

18.1 Complete Architecture Diagram

INPUT IMAGE $\mathbf{x}$

Backbone + FPN

IMAGE FEATURE MAP $\mathbf{F}$

conditioning

DIFFUSION DETECTOR

$\mathbf{B}_T \sim \text{Gaussian random boxes}$

Noisy proposal $\mathbf{B}_t$

Oriented RoI extraction

Feature Alignment

Fixed-size RoI features

Detection Head
 / \
 / \
 Classification Box refinement

Noise prediction

Accelerated diffusion

cleaner B

repeat

FINAL ORIENTED BOX
 (x,y,w,h,,)

FOUR OBJECT VERTICES

19 The Five Most Important Equations

19.1 Equation 1: Oriented Box Representation

$$\boxed{B = (x, y, w, h, \Delta\alpha, \Delta\beta)} \quad (138)$$

19.2 Equation 2: Four Vertices

$$\boxed{\begin{aligned} P_1 &= (x + \Delta\alpha, y - h/2), \\ P_2 &= (x + w/2, y + \Delta\beta), \\ P_3 &= (x - \Delta\alpha, y + h/2), \\ P_4 &= (x - w/2, y - \Delta\beta) \end{aligned}} \quad (139)$$

19.3 Equation 3: Forward Diffusion

$$\boxed{B_t = \alpha_t B_0 + \sigma_t \epsilon} \quad (140)$$

19.4 Equation 4: Feature Alignment

$$F'_c(i, j) = \frac{1}{n} \sum_{(x, y) \in \text{area}(i, j)} F_c(R(x, y, \theta)) \quad (141)$$

19.5 Equation 5: Accelerated Diffusion

$$\tilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \sigma_{t_i} (e^{h_i} - 1) \epsilon_\theta \quad (142)$$

with:

$$h_i = \lambda_{t_i} - \lambda_{t_{i-1}} \quad (143)$$

and:

$$\lambda_t = \log \frac{\alpha_t}{\sigma_t} \quad (144)$$

20 Summary of Part 1

This completes Part 1 of the survey, covering:

1. The problem of oriented object detection
2. Horizontal vs oriented bounding boxes
3. The six-parameter representation $(x, y, w, h, \Delta\alpha, \Delta\beta)$
4. The four vertices derivation
5. Feature alignment mathematics
6. Bilinear interpolation
7. The complete architecture pipeline
8. The five most important equations

21 The Diffusion Process on Bounding Boxes

21.1 Forward Diffusion Equation

Suppose the ground-truth box is B_0 . We want to corrupt it gradually:

$$B_0 \rightarrow B_1 \rightarrow B_2 \rightarrow \cdots \rightarrow B_T \quad (145)$$

At every step, more noise is added.

21.2 General Forward Diffusion Equation

A standard diffusion process can be written as:

$$\boxed{B_t = \alpha_t B_0 + \sigma_t \epsilon} \quad (146)$$

where:

$$\epsilon \sim \mathcal{N}(0, I) \quad (147)$$

Here:

- B_0 : clean box
- B_t : noisy box at time t
- α_t : signal scale
- σ_t : noise scale
- ϵ : Gaussian noise

21.3 What α_t Does

Consider:

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (148)$$

If $\alpha_t \approx 1$ and $\sigma_t \approx 0$, then:

$$B_t \approx B_0 \quad (149)$$

So the box is almost clean.
As t increases, typically:

$$\alpha_t \downarrow \quad (150)$$

and:

$$\sigma_t \uparrow \quad (151)$$

Then the original box becomes less visible.
At the final timestep:

$$B_T \approx \sigma_T \epsilon \quad (152)$$

Therefore:

$$\boxed{B_T \approx \text{Gaussian noise}} \quad (153)$$

21.4 The Cosine Schedule

The paper says that α_t follows a monotonically decreasing cosine schedule.

The basic idea is:

$$\alpha_t = \text{cosine-shaped function of } t \quad (154)$$

A common diffusion cosine schedule uses something of the form:

$$\bar{\alpha}_t = \cos^2 \left(\frac{t/T + s_0}{1 + s_0} \frac{\pi}{2} \right) \quad (155)$$

where s_0 is a small offset.

Then the signal scale can be related to $\sqrt{\bar{\alpha}_t}$.

The exact schedule in a particular implementation may differ, so the safest interpretation is:

$$\boxed{t \uparrow \Rightarrow \alpha_t \downarrow} \quad (156)$$

and therefore the box becomes increasingly noisy.

21.5 Why the Schedule is Necessary

We need a controlled transition:

$$\text{clean box} \rightarrow \text{noise} \quad (157)$$

If we suddenly destroyed the box, the network would have difficulty learning the reverse process.

Instead:

$$B_0 \rightarrow B_1 \rightarrow B_2 \rightarrow \cdots \rightarrow B_T \quad (158)$$

gives the model many intermediate difficulty levels.

22 Training Process

22.1 Training Steps

Training starts with the ground-truth box:

$$B_0 = (x, y, w, h, \Delta\alpha, \Delta\beta) \quad (159)$$

A random timestep t is selected.

Then we generate:

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (160)$$

The model receives:

$$\boxed{B_t, \quad F, \quad t} \quad (161)$$

where:

$$F = E(x) \quad (162)$$

The model then tries to determine how to denoise B_t .

22.2 What the Network Learns

Conceptually, it learns:

$$\boxed{(B_t, F, t) \rightarrow \epsilon_\theta(B_t, F, t)} \quad (163)$$

where ϵ_θ is the neural network's prediction of the noise.

The network therefore learns how to distinguish:

"This noisy box is partly random; given the image, which direction should I move it to recover an object box?"

22.3 Why the Image Helps

Imagine the noisy box is located around a region. The image features may contain an object (airplane, ship, etc.). The model can therefore determine:

- where the object is
- how large it is
- its orientation
- which category it belongs to

and use this information to denoise the box.

So:

$$\boxed{\text{image features tell the model what object geometry is plausible}} \quad (164)$$

23 Inference Process

23.1 Starting Point

Now we don't have a ground-truth box. So we start with:

$$\boxed{B_T \sim \mathcal{N}(0, I)} \quad (165)$$

or the appropriate Gaussian distribution over box variables.

23.2 Reverse Process

Then:

$$\boxed{B_T \rightarrow B_{T-1} \rightarrow B_{T-2} \rightarrow \cdots \rightarrow B_0} \quad (166)$$

At every step:

$$\boxed{B_t + \text{image features} \rightarrow \text{denoised box}} \quad (167)$$

23.3 Random Boxes Are Not Extra Information

When the paper says "The model starts with boxes sampled from a Gaussian distribution," it does not mean the user supplies random boxes. They are generated internally.

Therefore:

$$\boxed{\text{Input to detector} = \text{image}} \quad (168)$$

The random boxes are an internal starting point for diffusion.

24 Reverse Process Mathematics

24.1 Forward Process

$$\boxed{B_0 \rightarrow B_t} \quad (169)$$

24.2 Reverse Process

$$\boxed{B_t \rightarrow B_0} \quad (170)$$

24.3 Network Noise Prediction

The network estimates the noise:

$$\epsilon_\theta(B_t, t, F) \quad (171)$$

24.4 Deriving the Clean Box Estimate

If:

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (172)$$

then rearranging gives:

$$\alpha_t B_0 = B_t - \sigma_t \epsilon \quad (173)$$

Therefore:

$$\boxed{B_0 = \frac{B_t - \sigma_t \epsilon}{\alpha_t}} \quad (174)$$

If we don't know the actual noise ϵ , the neural network gives an estimate:

$$\epsilon_\theta(B_t, t, F) \quad (175)$$

Thus:

$$\boxed{\hat{B}_0 = \frac{B_t - \sigma_t \epsilon_\theta(B_t, t, F)}{\alpha_t}} \quad (176)$$

24.5 Meaning of the Denoising Network

The network isn't magically saying "Here is the final box." It is learning the noise direction.

Given B_t , it predicts ϵ_θ . Then the diffusion mathematics uses that prediction to move toward a cleaner box.

So:

$$\boxed{\text{network predicts noise} \rightarrow \text{diffusion equation removes noise}} \quad (177)$$

25 Why Many Steps Are Needed

Suppose we start from completely random boxes. That is very far from the correct object.

Going directly:

$$B_T \rightarrow B_0 \quad (178)$$

is difficult.

Instead:

$$\boxed{B_T \rightarrow B_{T-1} \rightarrow B_{T-2} \rightarrow \cdots \rightarrow B_0} \quad (179)$$

Each step is relatively small. This makes the generation process easier to learn.

26 The Problem: Inference is Slow

Suppose diffusion uses $T = 1000$ timesteps. Then naïve inference may require hundreds or thousands of network evaluations.

For object detection, that's expensive.

This motivates the paper's accelerated diffusion section.

27 Accelerated Diffusion

27.1 The Basic Idea

The paper says it uses an accelerated diffusion method based on the semi-linear structure of diffusion ODEs.

This is essentially in the family of DPM-Solver-type acceleration.

The basic idea is:

Instead of taking every tiny diffusion step:

$$T \rightarrow T - 1 \rightarrow T - 2 \rightarrow \cdots \rightarrow 0 \quad (180)$$

we can select a much smaller number of time points:

$$t_0 = T, t_1, t_2, \dots, t_M = 0 \quad (181)$$

For example, conceptually:

$$1000 \rightarrow 750 \rightarrow 500 \rightarrow 250 \rightarrow 0 \quad (182)$$

So we can take large steps while still approximating the continuous diffusion trajectory.

27.2 Diffusion as an ODE

A diffusion process can be described continuously using a time-dependent differential equation:

$$\boxed{\frac{dx_t}{dt} = f(t)x_t + g(t)\epsilon_\theta(x_t, t)} \quad (183)$$

or an equivalent formulation depending on the parameterization.

The important thing is that the equation contains:

1. a part involving x_t
2. a part involving the neural network

This gives a semi-linear ODE.

27.3 What Semi-Linear Means

A linear ODE has the form:

$$\frac{dx}{dt} = A(t)x \quad (184)$$

A nonlinear equation could be:

$$\frac{dx}{dt} = f(x, t) \quad (185)$$

A semi-linear equation can be written roughly as:

$$\boxed{\frac{dx}{dt} = A(t)x + N(x, t)} \quad (186)$$

where the first part is linear in x , while N is nonlinear.

Diffusion ODEs can be expressed in this useful structure. That means we can solve the linear portion analytically and approximate only the difficult nonlinear portion.

This is the mathematical motivation for DPM-Solver-style methods.

28 The Paper's Accelerated Diffusion Equation

28.1 Equation (6)

The paper gives:

$$\boxed{\tilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\epsilon_\theta(\tilde{x}_{t_{i-1}}, t_{i-1})} \quad (187)$$

with:

$$\boxed{h_i = \lambda_{t_i} - \lambda_{t_{i-1}}} \quad (188)$$

and the time points decrease:

$$t_0 = T > t_1 > \dots > t_M = 0 \quad (189)$$

28.2 What is $\tilde{x}_{t_0}$?

The initial value is:

$$\tilde{x}_{t_0} = x_T \quad (190)$$

and x_T is sampled from the Gaussian distribution.

For object detection, x corresponds to the box representation.

So more specifically:

$$\boxed{x_T \equiv B_T} \quad (191)$$

28.3 What are t_i ?

Instead of evaluating every integer timestep:

$$T, T-1, T-2, \dots, 0 \quad (192)$$

we choose:

$$\boxed{t_0, t_1, \dots, t_M} \quad (193)$$

where:

$$t_0 = T \quad (194)$$

and:

$$t_M = 0 \quad (195)$$

For example:

$$t_0 = 1000, t_1 = 700, t_2 = 400, t_3 = 100, t_4 = 0 \quad (196)$$

Then only five sampling points are used.

28.4 What is λ_t ?

The diffusion literature commonly defines the log signal-to-noise ratio as:

$$\boxed{\lambda_t = \log \frac{\alpha_t}{\sigma_t}} \quad (197)$$

or an equivalent log-SNR parameterization depending on notation.

Therefore:

$$\lambda_{t_i} = \log \frac{\alpha_{t_i}}{\sigma_{t_i}} \quad (198)$$

Then:

$$h_i = \lambda_{t_i} - \lambda_{t_{i-1}} \quad (199)$$

So:

$$\boxed{h_i = \Delta \lambda} \quad (200)$$

is simply the change in the log-SNR variable between two selected diffusion times.

28.5 Why Use λ Instead of t ?

Because diffusion dynamics are more naturally expressed in terms of the signal-to-noise ratio.

Recall:

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (201)$$

The relative strength of signal versus noise is roughly:

$$\frac{\alpha_t}{\sigma_t} \quad (202)$$

Taking logarithm gives:

$$\lambda_t = \log \frac{\alpha_t}{\sigma_t} \quad (203)$$

This gives a convenient coordinate system for solving the diffusion ODE.

28.6 Understanding $e^{h_i} - 1$

Since:

$$h_i = \lambda_{t_i} - \lambda_{t_{i-1}} \quad (204)$$

we have:

$$e^{h_i} = e^{\lambda_{t_i} - \lambda_{t_{i-1}}} \quad (205)$$

Therefore:

$$e^{h_i} = \frac{e^{\lambda_{t_i}}}{e^{\lambda_{t_{i-1}}}} \quad (206)$$

Since:

$$e^{\lambda_t} = \frac{\alpha_t}{\sigma_t} \quad (207)$$

we get:

$$e^{h_i} = \frac{\alpha_{t_i}/\sigma_{t_i}}{\alpha_{t_{i-1}}/\sigma_{t_{i-1}}} \quad (208)$$

or:

$$e^{h_i} = \frac{\alpha_{t_i}\sigma_{t_{i-1}}}{\alpha_{t_{i-1}}\sigma_{t_i}} \quad (209)$$

Thus $e^{h_i} - 1$ is determined entirely by the change in the diffusion schedule between the two selected timesteps.

28.7 Understanding Equation (6) Intuitively

Let's rewrite it conceptually:

$$\tilde{x}_{t_i} = \underbrace{\frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}}}_{\text{scaled current box}} - \underbrace{\sigma_{t_i}(e^{h_i} - 1)\epsilon_\theta}_{\text{denoising correction}} \quad (210)$$

So there are two major components:

28.7.1 Part 1

$$\frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}} \quad (211)$$

rescales the current state according to the new diffusion time.

28.7.2 Part 2

$$\sigma_{t_i}(e^{h_i} - 1)\epsilon_\theta \quad (212)$$

uses the network's noise prediction to correct the state toward a cleaner solution.

Therefore:

$$\boxed{\text{next state} = \text{scheduled transformation} - \text{predicted-noise correction}} \quad (213)$$

28.8 Why the Equation Subtracts the Predicted Noise

Suppose:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \quad (214)$$

The noisy part is $\sigma_t \epsilon$.

If the model predicts $\epsilon_\theta \approx \epsilon$, then subtracting an appropriately scaled version of ϵ_θ removes the noise contribution.

Thus:

$$\boxed{\text{predicted noise} \rightarrow \text{noise removal}} \quad (215)$$

28.9 Why Can We Skip Many Timesteps?

The crucial mathematical idea is that Equation (6) isn't just saying "Let's arbitrarily jump over steps."

Instead, it is derived from an approximation to the continuous diffusion ODE.

The exact ODE solution involves an integral:

$$x(t_i) = \text{analytical linear evolution} + \int_{t_{i-1}}^{t_i} \text{weighted noise prediction } dt \quad (216)$$

The solver approximates this integral.

Therefore a large step can approximate the cumulative effect of many small steps.

That is why it can accelerate inference.

28.10 Why This is Different from Simply Increasing Step Size

If you simply skipped 900 steps randomly, you would generally lose accuracy.

DPM-Solver-type methods exploit the mathematical structure:

diffusion ODE has a special semi-linear form

(217)

and analytically account for the linear component. Then they approximate the neural-network-dependent integral.

So the acceleration is mathematically motivated.

28.11 What "Exponentially Weighted Integral" Means

The exact solution of the semi-linear diffusion ODE contains an integral of the form:

$$\int e^{\text{something}} \epsilon_{\theta}(\cdot) d\lambda \quad (218)$$

The exponential term weights the contribution of the network's prediction according to the diffusion state.

The solver approximates this integral efficiently.

That is the deeper reason the paper says:

"exponentially weighted integral of the noise prediction model."

29 Putting Accelerated Diffusion into Object Detection

29.1 Inference with Accelerated Diffusion

At inference:

$$B_T \sim \mathcal{N}(0, I) \quad (219)$$

Then:

$$\boxed{B_T \rightarrow B_{t_1} \rightarrow B_{t_2} \rightarrow \cdots \rightarrow B_0} \quad (220)$$

At each step:

1. Take current noisy box.
2. Crop corresponding features.
3. Feature-align them.
4. Feed them to detection head.
5. Predict class and box/noise information.
6. Apply accelerated diffusion update.
7. Obtain the next, cleaner box.

29.2 The Complete Inference Pipeline

The complete process is approximately:

```

Image
↓
Backbone + FPN
↓
Feature map F

while simultaneously:
    B_T ~ N(0,I)

Then:
    (F, B_T, t_T)
    ↓
    RoI extraction
    ↓
    Feature alignment
    ↓
    Detection head
    ↓
    , class information, box refinement
    ↓
    accelerated diffusion
    ↓
    B_{t_{T-1}}

Repeat.

Eventually:
    B_0

```

30 Training vs Inference Distinction

30.1 Training

Training starts with a real ground-truth box:

$$B_0 \tag{221}$$

and adds noise:

$$B_0 \rightarrow B_t \tag{222}$$

The image goes through the encoder:

$$x \rightarrow F \tag{223}$$

Then:

$$(B_t, F, t) \rightarrow \epsilon_\theta \tag{224}$$

The predicted noise is compared with the actual noise:

$$\epsilon_\theta \approx \epsilon \tag{225}$$

The model learns the reverse process.

So training can be summarized as:

$$\boxed{\text{Ground-truth box} \rightarrow \text{add noise} \rightarrow \text{noisy box}} \tag{226}$$

and the network learns:

$$\boxed{\text{noisy box} + \text{image} \rightarrow \text{denoising direction}} \tag{227}$$

30.2 Training vs Inference Table

Table 3: Training vs Inference

Training	Inference
Ground-truth box available	Ground-truth unavailable
Start from B_0	Start from B_T
Add noise	Remove noise
$B_0 \rightarrow B_t$	$B_T \rightarrow B_0$
Learn reverse process	Use learned reverse process
Image provides condition	Image provides condition

So:

$$\boxed{\text{training} = \text{learn to reverse corruption}} \tag{228}$$

while:

$$\boxed{\text{inference} = \text{actually perform the reverse process}} \tag{229}$$

31 The Four Major Problems and Four Solutions

You can remember the paper through this table:

Table 4: Problems and Solutions in OrientedDiffDet

Problem	Solution
Objects have different scales	FPN
Objects have arbitrary orientations	Oriented box representation
Horizontal RoI is poorly aligned with rotated object	Feature alignment
Need proposals without manually designed rotated anchors	Diffusion-based box generation
Diffusion sampling is slow	Accelerated diffusion / DPM-Solver

32 The Complete Mathematical Formulation

32.1 Step 1: Image Encoding

$$F = E(x) \quad (230)$$

where E is backbone + FPN.

32.2 Step 2: Ground-Truth Oriented Box

$$B_0 = (x, y, w, h, \Delta\alpha, \Delta\beta) \quad (231)$$

32.3 Step 3: Generate Vertices

$$\begin{aligned} P_1 &= \left(x + \Delta\alpha, y - \frac{h}{2}\right) \\ P_2 &= \left(x + \frac{w}{2}, y + \Delta\beta\right) \\ P_3 &= \left(x - \Delta\alpha, y + \frac{h}{2}\right) \\ P_4 &= \left(x - \frac{w}{2}, y - \Delta\beta\right) \end{aligned} \quad (232)$$

32.4 Step 4: Add Diffusion Noise

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (233)$$

with:

$$\epsilon \sim \mathcal{N}(0, I) \quad (234)$$

32.5 Step 5: Extract Oriented RoI

Project the oriented proposal onto F :

$$B_t \rightarrow R_t \quad (235)$$

32.6 Step 6: Feature Alignment

Sample the feature map at rotated coordinates:

$$F'_c(i, j) = \frac{1}{n} \sum_{(x, y) \in \text{area}(i, j)} F_c(R(x, y, \theta)) \quad (236)$$

32.7 Step 7: Detection Head

$$F' \rightarrow \text{FC layers} \rightarrow \begin{cases} \text{classification} \\ \text{box refinement} \end{cases} \quad (237)$$

32.8 Step 8: Noise Prediction

The network estimates:

$$\epsilon_\theta(B_t, F, t) \quad (238)$$

32.9 Step 9: Estimate Clean Box

From:

$$B_t = \alpha_t B_0 + \sigma_t \epsilon \quad (239)$$

we get:

$$\hat{B}_0 = \frac{B_t - \sigma_t \epsilon_\theta(B_t, F, t)}{\alpha_t} \quad (240)$$

32.10 Step 10: Accelerated Update

For selected times t_i :

$$\tilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \sigma_{t_i} (e^{h_i} - 1) \epsilon_\theta(\tilde{x}_{t_{i-1}}, t_{i-1}) \quad (241)$$

where:

$$h_i = \lambda_{t_i} - \lambda_{t_{i-1}} \quad (242)$$

and commonly:

$$\lambda_t = \log \frac{\alpha_t}{\sigma_t} \quad (243)$$

33 The Most Intuitive Interpretation of Equation (6)

Forget the complicated mathematics for a moment.

Suppose the current box is noisy. The image features tell the model where the actual object is. The neural network predicts the noise/error direction.

Then accelerated diffusion says:

"Given where we are in diffusion time, use this predicted noise and the known diffusion schedule to make a mathematically appropriate jump toward a cleaner box."

So:

$$\boxed{\text{current noisy box} + \text{image information} + \text{diffusion time} \rightarrow \text{next cleaner box}} \quad (244)$$

34 Important Distinction: Feature Alignment vs Diffusion

A more accurate statement is:

$$\boxed{\text{Feature alignment} \rightarrow \text{provides better rotation-aligned features}} \quad (245)$$

while:

$$\boxed{\text{Diffusion} \rightarrow \text{generates/refines the full oriented box representation}} \quad (246)$$

The diffusion process still has to generate:

$$x, y, w, h, \Delta\alpha, \Delta\beta \quad (247)$$

Therefore it still predicts orientation-related information.

Feature alignment simply makes the image features used by the detection decoder more suitable for arbitrarily oriented objects.

35 Complete Conceptual Picture

Here is the mental model to remember:

35.1 Input

$$\boxed{\text{Image}} \quad (248)$$

35.2 Image Encoder

$$\boxed{\text{Backbone} + \text{FPN} \rightarrow \text{multi-scale image features}} \quad (249)$$

35.3 Training

$$\text{Ground-truth oriented box } B_0 \rightarrow \text{add Gaussian noise} \rightarrow B_t \rightarrow \text{network learns to predict noise } \epsilon_\theta(B_t, F_t) \quad (250)$$

35.4 Inference

$$B_T \sim \mathcal{N}(0, I) \rightarrow B_T + \text{image features} \rightarrow \text{Feature alignment} \rightarrow \text{Detection head} \rightarrow \text{predicted noise/refinement} \quad (251)$$

36 Summary of Key Equations

Table 5: Key Equations in OrientedDiffDet

Equation	Description
B	= Oriented box representation
$(x, y, w, h, \Delta\alpha, \Delta\beta)$	
$P_1 = (x + \Delta\alpha, y - h/2)$	Vertex 1
$P_2 = (x + w/2, y + \Delta\beta)$	Vertex 2
$P_3 = (x - \Delta\alpha, y + h/2)$	Vertex 3
$P_4 = (x - w/2, y - \Delta\beta)$	Vertex 4
$B_t = \alpha_t B_0 + \sigma_t \epsilon$	Forward diffusion
$\epsilon \sim \mathcal{N}(0, I)$	Gaussian noise
$F'_c(i, j)$	= Feature alignment
$\frac{1}{n} \sum_{(x,y) \in \text{area}(i,j)} F_c(R(x, y, \theta))$	
$\tilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}}$	Accelerated diffusion
$\sigma_{t_i} (e^{h_i} - 1) \epsilon_\theta$	
$h_i = \lambda_{t_i} - \lambda_{t_{i-1}}$	Log-SNR difference
$\lambda_t = \log \frac{\alpha_t}{\sigma_t}$	Log-SNR
$\hat{B}_0 = \frac{B_t - \sigma_t \epsilon_\theta(B_t, F_t, t)}{\alpha_t}$	Clean box estimate
$\theta = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right)$	Angle calculation
$R(\theta)$	= Rotation matrix
$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$	
$x_s = x_c + u \cos \theta - v \sin \theta$	Rotated x-coordinate
$y_s = y_c + u \sin \theta + v \cos \theta$	Rotated y-coordinate

37 Conclusion

This comprehensive survey has covered all major aspects of OrientedDiffDet:

1. **The Problem:** Oriented object detection in remote-sensing images with objects at arbitrary orientations, different scales, and dense packing.

2. **Horizontal vs Oriented Boxes:** Horizontal boxes contain background and lose orientation information; oriented boxes tightly fit objects and preserve orientation.
3. **Six-Parameter Representation:** $(x, y, w, h, \Delta\alpha, \Delta\beta)$ where $\Delta\alpha, \Delta\beta$ control tilt, avoiding direct angle regression problems.
4. **Four Vertices:** $P_1 = (x + \Delta\alpha, y - h/2)$, $P_2 = (x + w/2, y + \Delta\beta)$, $P_3 = (x - \Delta\alpha, y + h/2)$, $P_4 = (x - w/2, y - \Delta\beta)$.
5. **Feature Alignment:** $F'_c(i, j) = \frac{1}{n} \sum_{(x,y) \in \text{area}(i,j)} F_c(R(x, y, \theta))$ — samples features along rotated RoI axes with bilinear interpolation.
6. **Forward Diffusion:** $B_t = \alpha_t B_0 + \sigma_t \epsilon$ — progressively adds noise to the ground-truth box.
7. **Reverse Diffusion:** $B_T \rightarrow B_{T-1} \rightarrow \dots \rightarrow B_0$ — progressively denoises random boxes guided by image features.
8. **Accelerated Diffusion:** $\tilde{x}_{t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} \tilde{x}_{t_{i-1}} - \sigma_{t_i} (e^{h_i} - 1) \epsilon_\theta$ with $h_i = \lambda_{t_i} - \lambda_{t_{i-1}}$ and $\lambda_t = \log(\alpha_t / \sigma_t)$ — allows large time jumps using ODE solver.
9. **Architecture:** Backbone + FPN for image features, oriented RoI extraction, feature alignment, detection head for classification/box refinement/noise prediction, and accelerated diffusion sampler.
10. **Key Insight:** Instead of relying on predefined anchors, start with random boxes and use conditional diffusion to generate oriented object boxes.

$$\boxed{\text{Traditional: predefined anchors} \rightarrow \text{regression} \rightarrow \text{final boxes}} \quad (252)$$

$$\boxed{\text{OrientedDiffDet: random boxes} + \text{image features} \xrightarrow{\text{diffusion}} \text{object boxes}} \quad (253)$$

The core innovation is treating object detection as a conditional generation problem where the diffusion process transforms random boxes into accurate oriented bounding boxes, guided by image features.